

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий

Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: Информационные системы и технологии

ОТЧЕТ

по проектной практике

Студент: Рыбакова Елена Андреевна Группа: 251-337

Место прохождения практики: Московский Политех, кафедра Информатика и
информационные технологии

Отчет принят с оценкой _____ Дата _____

Руководитель практики: Дагаев Александр Евгеньевич

Москва 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ

1. Общая информация о проекте:
 - Название проекта
 - Цели и задачи проекта
2. Общая характеристика деятельности организации
 - Наименование заказчика
 - Организационная структура
 - Описание деятельности
3. Описание задания по проектной практике
4. Описание достигнутых результатов по проектной практике

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

ПРИЛОЖЕНИЯ

ВВЕДЕНИЕ

1. Общая информация о проекте:

Название проекта: Magic Tetris.

Цели проекта: Реализовать игру на Python с использованием библиотеки PyGame; создать статичный сайт проекта; организовать публичный Git-репозиторий для демонстрации проекта и организации совместной работы.

Задачи проекта: Разработать Tetris с дополнительными механиками маны, протестировать игру, задать верстку 5-тистраничного сайта, добавить .css, добавить Markdown документ для репозитория, составить итоговый отчёт.

2. Общая характеристика деятельности организации

Наименование заказчика: Федеральное государственное автономное образовательное учреждение высшего образования «Московский политехнический университет»

Организационная структура: Организационная структура Московского политехнического университета (Московского политеха) включает несколько ключевых уровней управления, факультеты, институты, лаборатории, центры и филиалы.

Описание деятельности: Университет осуществляет образовательную, научную и проектную деятельности.

3. Описание задания по проектной практике

Проектная (учебная) практика направлена на закрепление навыков работы с инструментами контроля версий, технической документацией и основами веб-разработки, а также на получение опыта взаимодействия с профессиональным сообществом. Трудоёмкость практики составляет 72 академических часа. Задание состоит из обязательной базовой части и вариативной части, содержание которой определяется выпускающей кафедрой или куратором проекта.

Базовая часть задания

Базовая часть является обязательной для выполнения всеми студентами и включает следующие этапы:

1. Настройка Git и репозитория: Создание репозитория на платформе GitHub или GitVerse на основе предоставленного шаблона. Освоение базовых команд Git (клонирование, коммит, пуш, работа с ветками) и регулярная фиксация изменений с использованием осмысленных сообщений.

2. Написание документов в Markdown: Оформление всех материалов проекта (описаний, планов, журналов прогресса) с использованием облегчённого языка разметки Markdown.

3. Создание статического веб-сайта: Разработка веб-сайта проекта с использованием HTML и CSS либо генераторов статических сайтов (предпочтительно Hugo). Сайт должен обладать уникальным оформлением (совпадение с другими работами не более 50%) и содержать следующие разделы:

- Домашняя страница с аннотацией.
- Страница «О проекте».
- Раздел «Участники» с описанием личного вклада.
- Лента «Журнал» (минимум три поста о прогрессе).
- Страница «Ресурсы» со ссылками на полезные материалы и партнёров.
- Графические и медиа-материалы, иллюстрирующие проект.

4. Взаимодействие с организацией-партнёром: Посещение мероприятий, организация встреч или участие в стажировках, согласованных с куратором. Подготовка отчёта о полученном опыте и его связи с проектными задачами.

5. Подготовка отчётной документации: Формирование итогового отчёта по практике по установленному шаблону в форматах DOCX и PDF, размещение файлов в репозитории и загрузка в систему дистанционного обучения (СДО).

Вариативная часть задания

Разработать игровой продукт на языке Python с применением библиотеки PyGame.

4. Описание достигнутых результатов по проектной практике

В ходе выполнения проектной (учебной) практики командой в составе трёх человек были полностью решены поставленные задачи базовой и вариативной частей задания. Трудоёмкость практики (72 академических часа) соблюдена.

4.1. Результаты базовой части

- Репозиторий и Git: Создан публичный Git-репозиторий на платформе [GitHub/GitVerse]. Ведётся история коммитов с осмысленными сообщениями. Освоены операции клонирования, ветвления и слияния.

- Документация в Markdown: Разработан комплект сопроводительной документации в формате Markdown: файл README.md с описанием проекта, журнал прогресса работ и технические заметки.

- Статический веб-сайт: Создан и наполнен контентом сайт проекта, содержащий все требуемые разделы: домашняя страница с аннотацией, страница «О проекте» (описание Magic Tetris), страница «Участники», лента новостей и страница «Ресурсы». Сайт обладает CSS-оформлением и необходимой информацией о проекте.

- Отчётная документация: Сформированы и загружены в СДО итоговые файлы отчёта.

4.2. Результаты вариативной части

- Участник 1 (Радов Глеб Андреевич): Разработка игровой логики «Magic Tetris».
 - Реализована основная механика тетриса (движение, поворот, коллизии, удаление линий).
 - Разработана и интегрирована уникальная система маны (накопление и расходование на заклинания “Замена”, “Пропуск” и “Замедление”).
 - Написан модуль подсчёта очков и визуализации интерфейса игры.
 - Создано техническое описание архитектуры кода с блок-схемой основных классов.
- Участник 2 (Попов Дмитрий Игоревич): Вёрстка веб-сайта и стили.

- Спроектирована структура сайта, выполнена вёрстка 5 страниц (HTML5).
- Разработаны стили (CSS3) с уникальным оформлением, обеспечена адаптивность.
- Осуществлена выгрузка сайта в репозиторий проекта.
- Создана структура репозитория и осуществление выгрузок.
- Участник 3 (Рыбакова Елена Андреевна): Тестирование и техническая документация.
 - Создание визуальных элементов Magic Tetris и консультация по оформлению сайта.
 - Написан итоговый отчёт по практике, оформлены все документы в формате Markdown.
 - Тестирование игрового продукта индивидуального задания “Magic Tetris”.

ЗАКЛЮЧЕНИЕ

В ходе проектной (учебной) практики командой в составе трёх студентов были успешно выполнены все поставленные задачи базовой и вариативной частей задания. Трудоемкость практики в объеме 72 академических часов соблюдена полностью.

По итогам практики получены следующие результаты:

1. Освоены инструменты командной разработки. Участники изучили и применили на практике базовые и продвинутые операции с системой контроля версий Git: создание репозитория, клонирование, ветвление, слияние веток и публикация изменений. Вся история разработки проекта сохранена в публичном репозитории и сопровождается осмысленными сообщениями к коммитам. Получен опыт параллельной работы нескольких человек над одним проектом без возникновения критических конфликтов в кодовой базе.

Ссылка: [GitHub - Eternal-ded/Praktika-proerktna · GitHub](https://github.com/Eternal-ded/Praktika-proerktna).

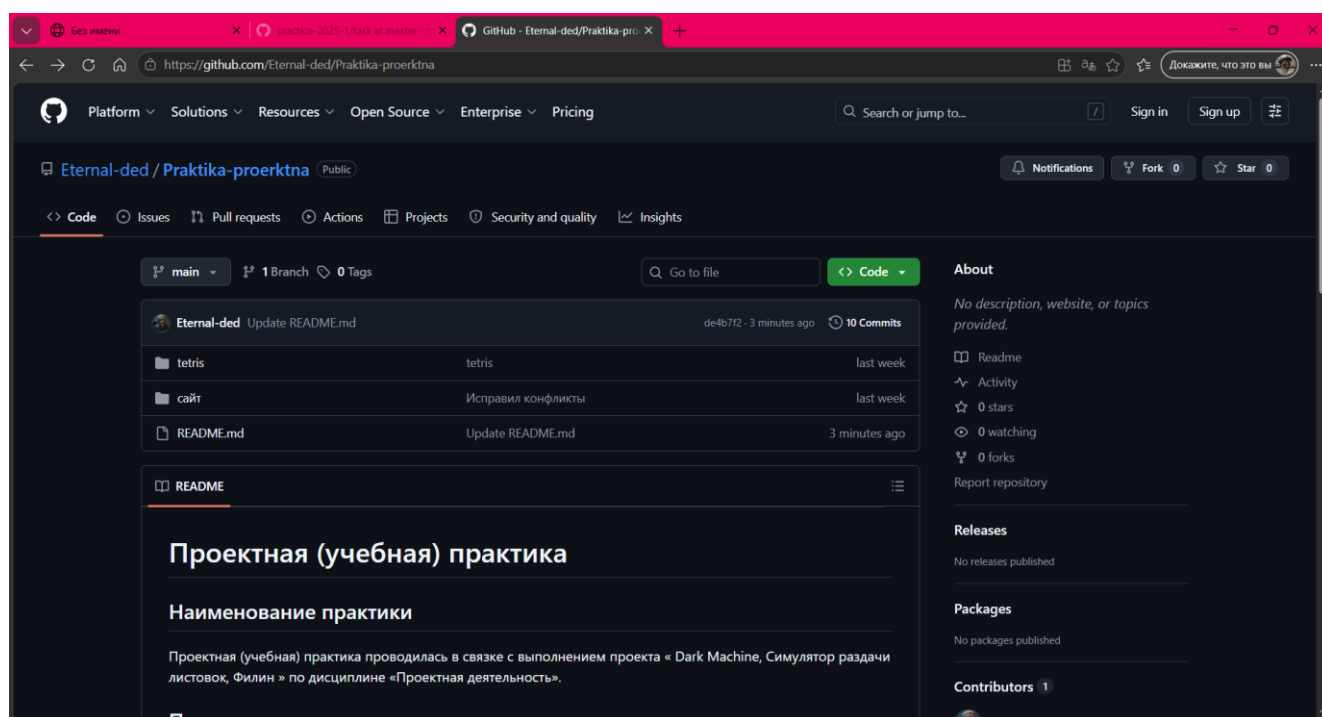


Рисунок 1 - репозиторий Git

2. Разработан игровой программный продукт «Magic Tetris». На языке Python с использованием библиотеки PyGame реализован функционирующий прототип игры, включающий как классические механики тетриса (движение фигур, определение коллизий, удаление заполненных линий), так и оригинальную авторскую механику — систему маны с заклинаниями «Замена», «Пропуск» и «Замедление». Продукт прошёл внутреннее тестирование и продемонстрировал стабильную работу. Практическая ценность проекта заключается в закреплении навыков объектно-ориентированного программирования, работы с игровыми циклами и обработки событий в PyGame.

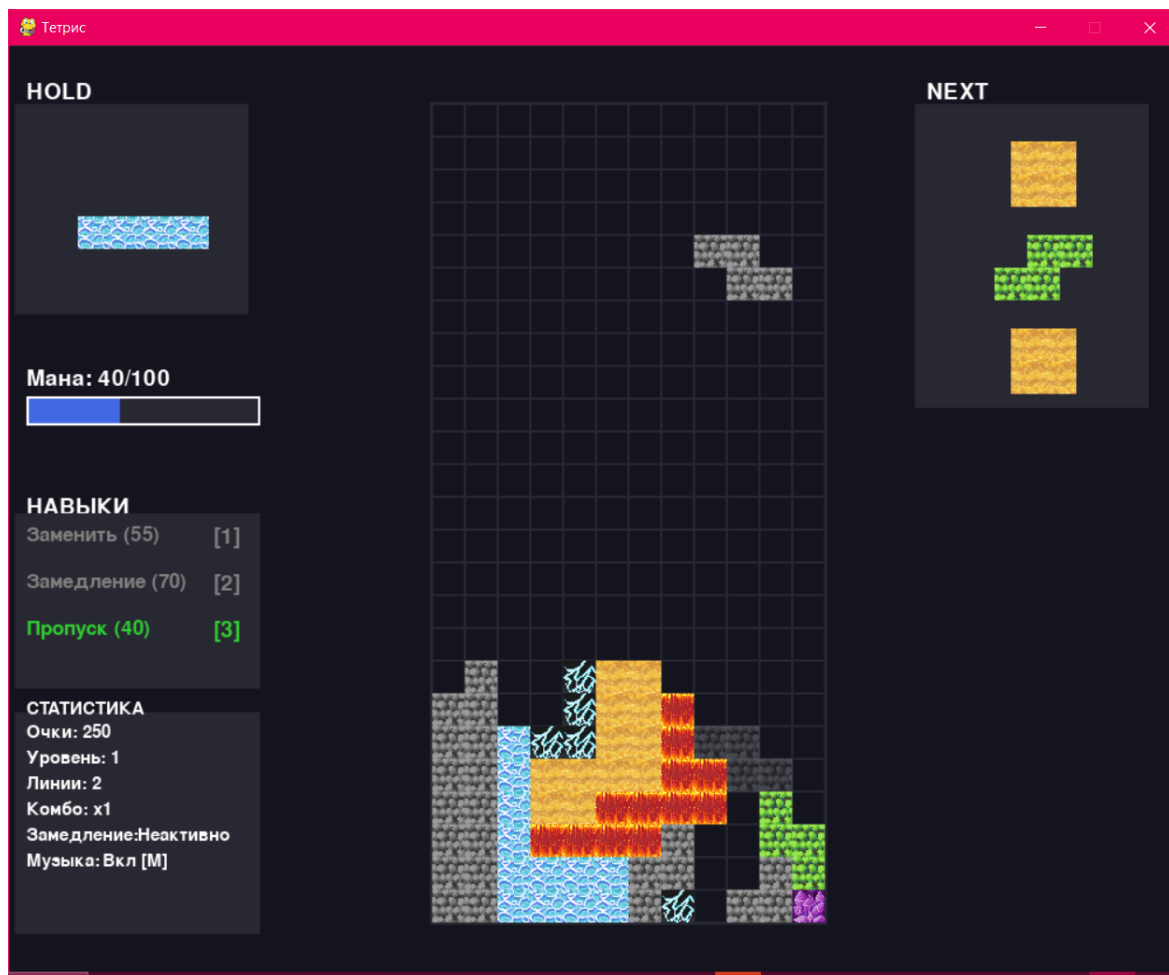


Рисунок 2 - тетрис

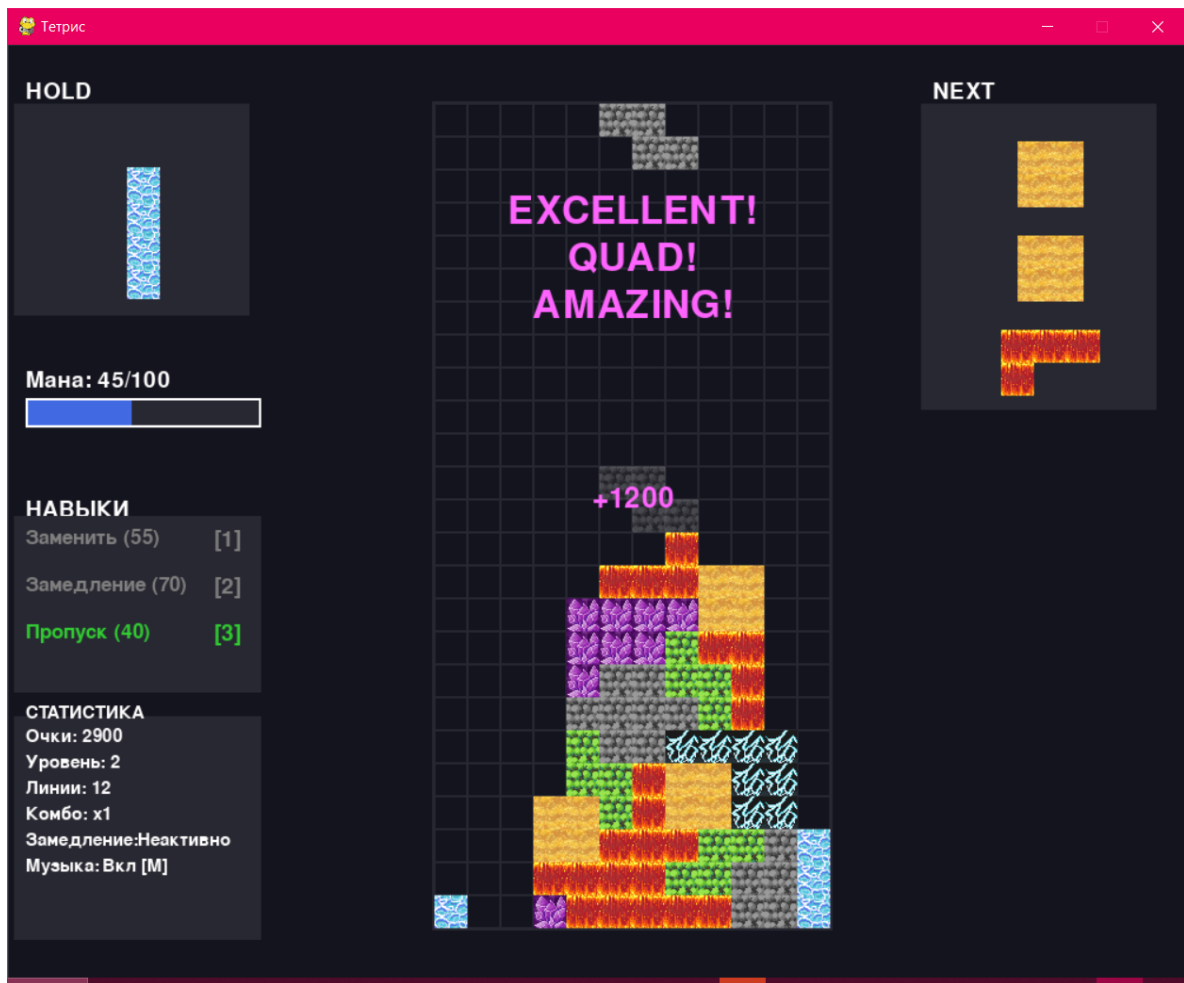


Рисунок 3 - тетрис комбо

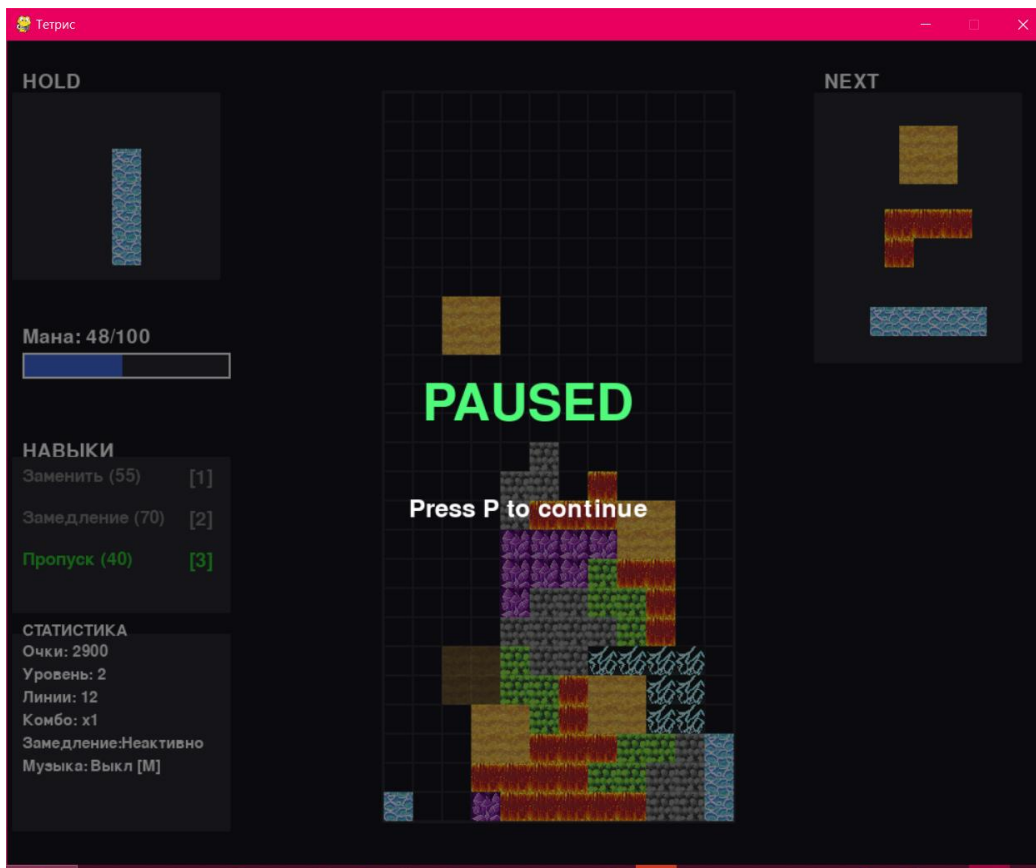


Рисунок 4 - тетрис пауза

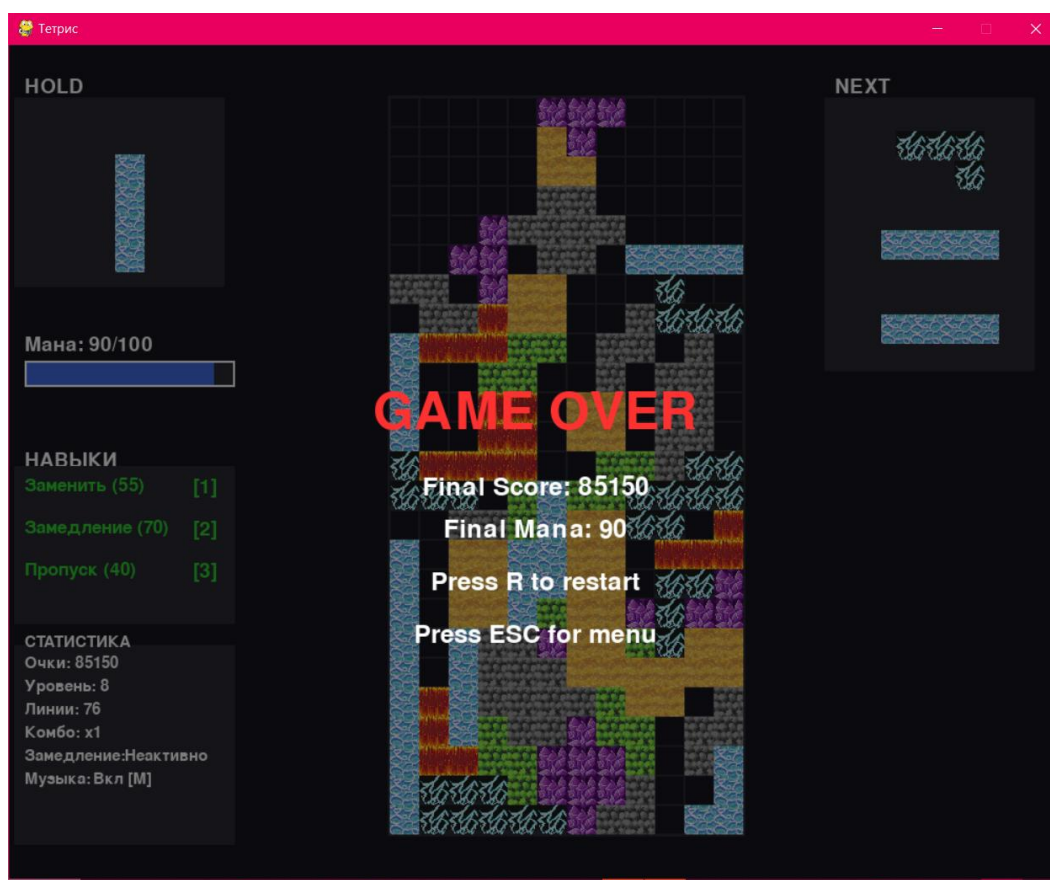


Рисунок 5 - тетрис конец игры

3. Создан статический веб-сайт проекта. Средствами HTML5 и CSS3 свёрстан и наполнен контентом сайт, включающий все требуемые разделы: домашнюю страницу, страницы «О проекте», «Участники», «Журнал прогресса» и «Ресурсы». Сайт обладает уникальным визуальным оформлением и содержит скриншоты игрового процесса, иллюстрирующие выполненную работу. В процессе создания сайта получены и закреплены навыки семантической вёрстки, каскадных стилей и размещения статического контента.

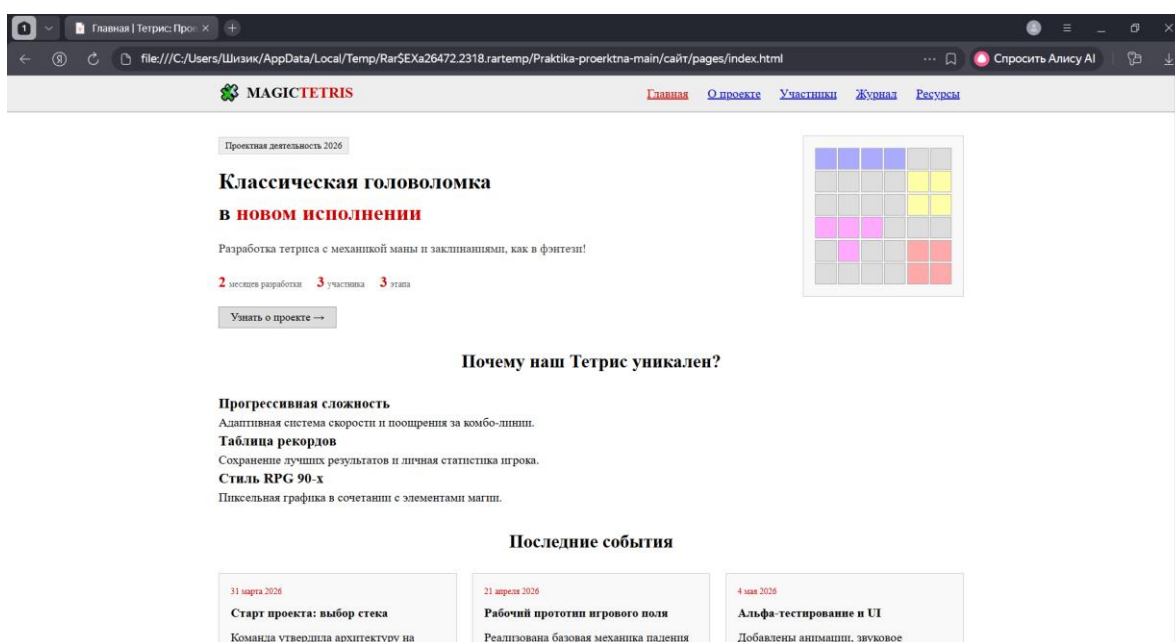


Рисунок 6 - index

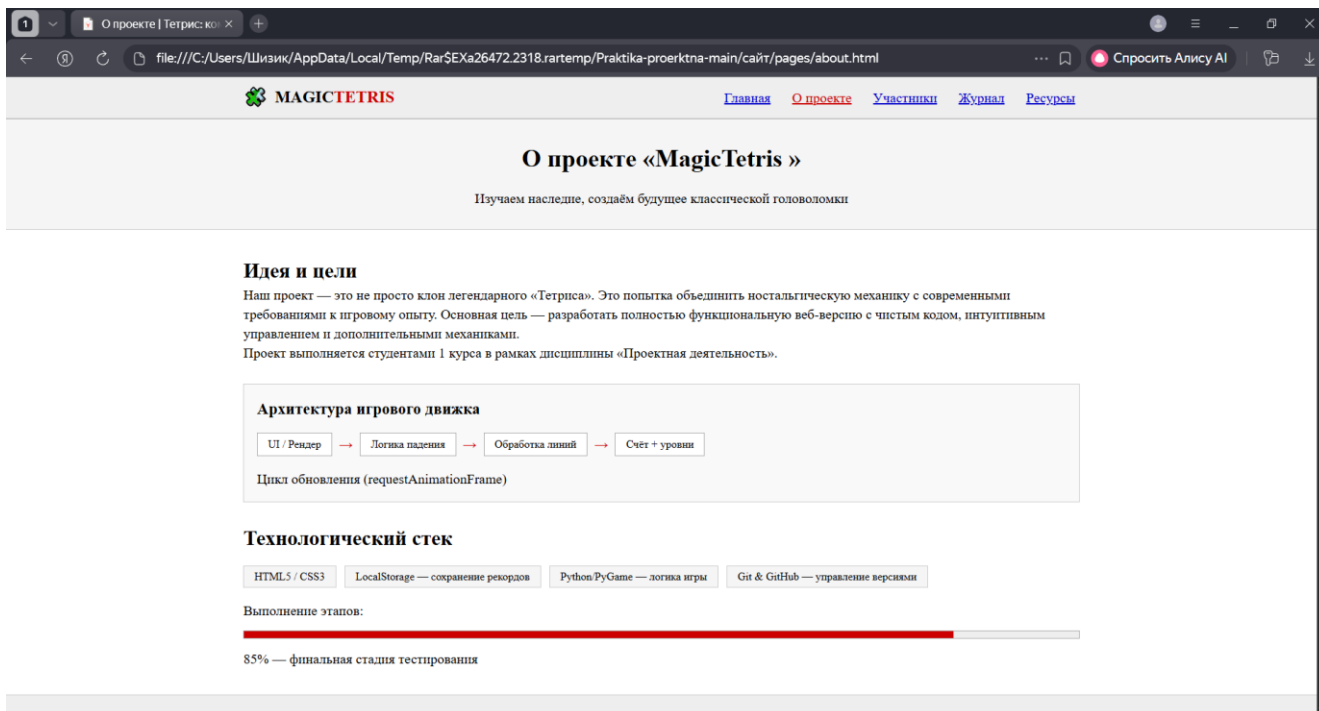


Рисунок 7 - about

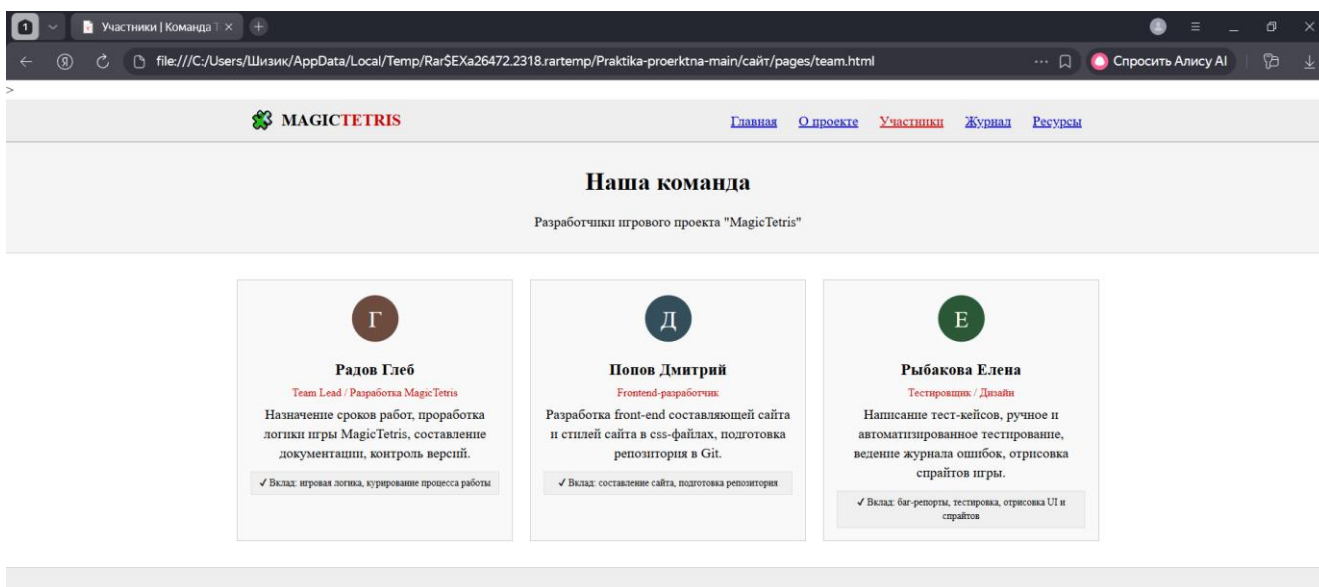


Рисунок 8 - team

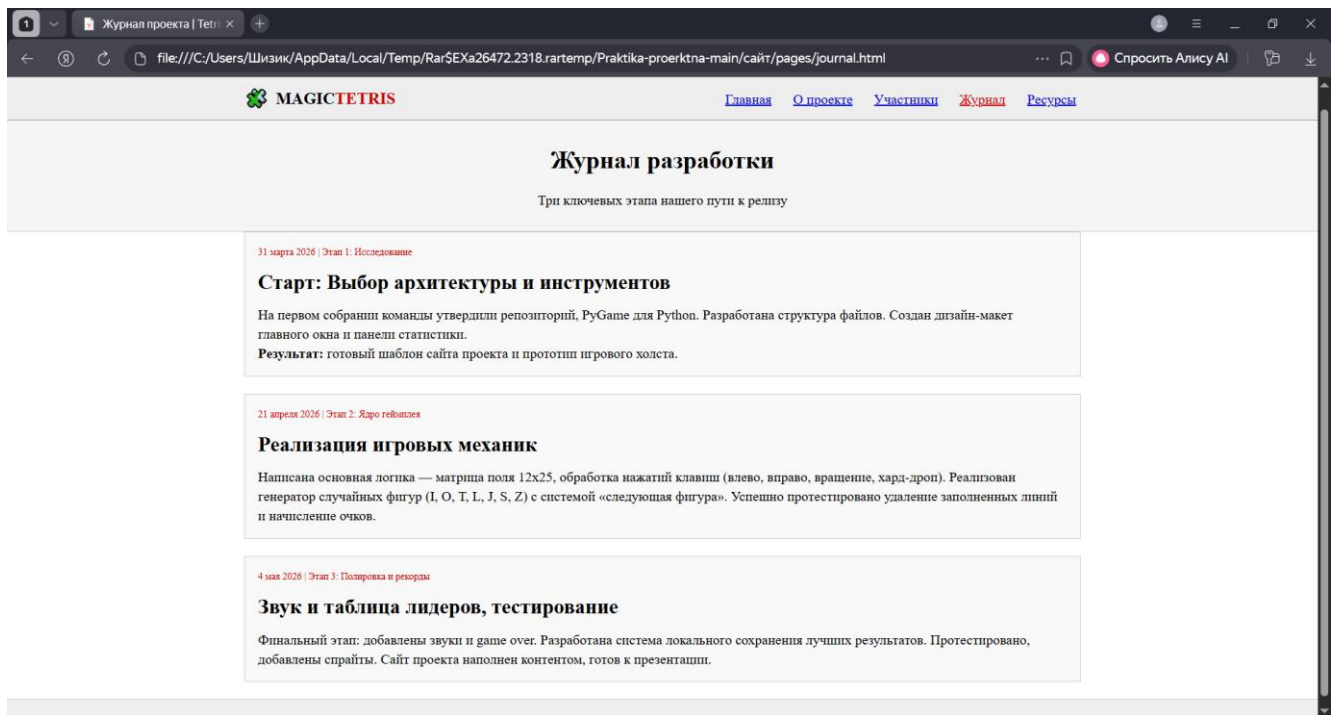


Рисунок 9 - journal

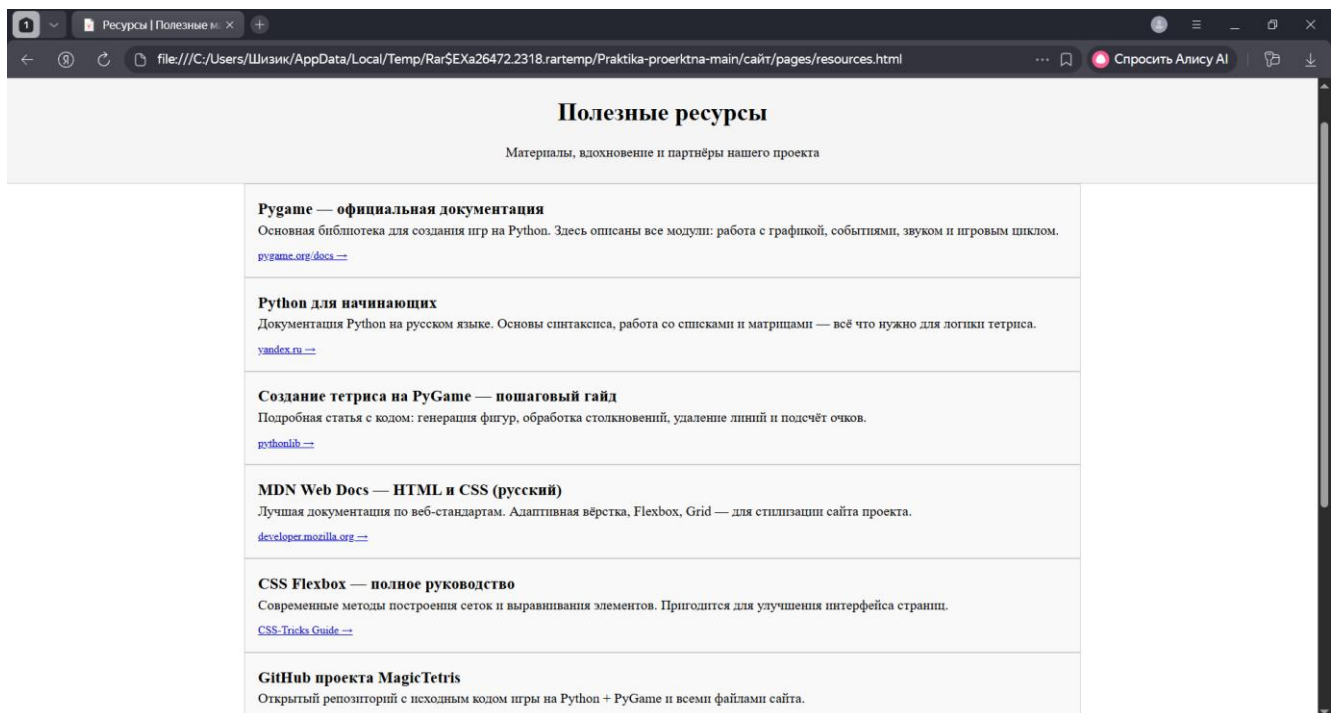


Рисунок 10 - resources

4. Оформлена техническая и отчётная документация. На языке разметки Markdown подготовлено описание проекта, правила игры и руководство пользователя. Сформирован итоговый отчёт по практике, включающий полную информацию о проекте, ходе работ, распределении личного вклада участников и достигнутых результатах. Отчёт предоставлен в требуемых форматах DOCX и PDF и загружен в систему дистанционного обучения.

 README

Проектная (учебная) практика

Наименование практики

Проектная (учебная) практика проводилась в связке с выполнением проекта « Dark Machine, Симулятор раздачи листовок, Филин » по дисциплине «Проектная деятельность».

Период проведения

С 1 апреля 2026 г. по 21 мая 2026 г.

Участники (ФИО)

ФИО	Учебная группа	Код направления подготовки	Профиль образовательной программы
Радов Глеб Андреевич	251-337	09.03.02	Информационные системы и технологии
Попов Дмитрий Игоревич	251-337	09.03.02	Информационные системы и технологии
Рыбакова Елена Андреевна	251-337	09.03.02	Информационные системы и технологии

Рисунок 11 - README.md

5. Оценка личного вклада участников. Все члены команды внесли значимый вклад в общий результат:

- Участник 1 реализовал основную и дополнительную игровые механики, интерфейс игры и техническое описание архитектуры кода.
- Участник 2 спроектировал и сверстал веб-сайт, разработал CSS-стили, организовал структуру репозитория и выгрузку материалов.
- Участник 3 выполнила тестирование игрового продукта, участвовала в создании визуальных элементов, а также подготовила итоговую отчётную и сопроводительную документацию.

Вывод: Проектная практика позволила участникам в условиях, приближенных к реальной командной разработке, закрепить компетенции, предусмотренные учебным планом. Получен завершённый программный продукт и сопутствующие артефакты (веб-сайт, документация). Ценность выполненных задач для заказчика Московский политехнический университет состоит в демонстрации студентами способности вести полный цикл проекта — от идеи до публичной презентации результата, — используя современные инструменты разработки и документирования. Команда готова к дальнейшему развитию проекта и участию в новых проектных активностях.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Введение в CSS верстку:

https://developer.mozilla.org/ru/docs/Learn_web_development/Core/CSS_layout/Introduction

2. DevTools для «чайников»: <https://habr.com/ru/articles/548898/>

3. Элементы

HTML: <https://developer.mozilla.org/ru/docs/Web/HTML/Element>

4. Основы

HTML: https://developer.mozilla.org/ru/docs/Learn_web_development/Getting_started/Your_first_website/Creating_the_content

5. Основы CSS: <https://developer.mozilla.org/ru/docs/Web/CSS>

6. <https://doka.guide/>

7. Официальная документация Git: <https://git-scm.com/book/ru/v2>

8. https://skillbox.ru/media/code/что_такое_git_obyasnyаем_na_skhemakh/

9. Бесплатный курс на Hexlet по Git: https://ru.hexlet.io/courses/intro_to_git

10. Уроки по Markdown: https://ru.hexlet.io/lesson_filters/markdown

ПРИЛОЖЕНИЯ

1. tetris.py

```
import pygame
import sys
import random
import os

class TetrisGame:
    def __init__(self, screen_width, screen_height):
        self.game_over_music_playing = False
        self.screen_width = max(900, screen_width) # Минимальная ширина 900px
        self.screen_height = max(700, screen_height) # Минимальная высота 700px

        # Создаем окно с переданными размерами
        self.screen = pygame.display.set_mode((self.screen_width, self.screen_height))
        pygame.display.set_caption("Тетрис")

        # Инициализируем музыку
        try:
            pygame.mixer.init(frequency=44100, size=-16, channels=2, buffer=512)
        except:
            pygame.mixer.init()

        self.clock = pygame.time.Clock()
        self.fps = 60
        self.running = True
        self.paused = False
        self.game_over = False

        self.background_color = (20, 20, 30)
        self.text_color = (255, 255, 255)
        self.accent_color = (80, 250, 123)
        self.grid_color = (40, 40, 50)
        self.mana_color = (65, 105, 225)
        self.skill_available_color = (50, 205, 50)
        self.skill_unavailable_color = (128, 128, 128)
```

```
self.title_font = pygame.font.Font(None, 74)
self.info_font = pygame.font.Font(None, 36)
self.score_font = pygame.font.Font(None, 28)
self.combo_font = pygame.font.Font(None, 48)
self.skill_font = pygame.font.Font(None, 26)
self.stats_font = pygame.font.Font(None, 22) # Уменьшенный шрифт для статистики

# Размеры сетки
self.grid_width = 12
self.grid_height = 25

# Адаптивный размер блока
left_panel_width = 260
right_panel_width = 220
max_block_size_by_width = (self.screen_width - left_panel_width - right_panel_width -
40) // self.grid_width
max_block_size_by_height = (self.screen_height - 100) // self.grid_height
self.block_size = max(25, min(35, max_block_size_by_width,
max_block_size_by_height))

# Позиция сетки по центру
total_grid_width = self.grid_width * self.block_size
total_grid_height = self.grid_height * self.block_size

# Центрируем сетку между панелями и по вертикали
self.grid_x = left_panel_width + 10 + (self.screen_width - left_panel_width -
right_panel_width - total_grid_width) // 2
self.grid_y = (self.screen_height - total_grid_height) // 2

# Позиции боковых панелей
self.left_panel_x = 15
self.right_panel_x = self.screen_width - right_panel_width + 5

# Увеличиваем отступы между элементами
self.hold_x = self.left_panel_x
self.hold_y = self.grid_y + 20

self.mana_x = self.left_panel_x
```

```

self.mana_y = self.hold_y + 230 # Увеличен отступ

self.skills_x = self.left_panel_x
self.skills_y = self.mana_y + 110 # Увеличен отступ

self.stats_x = self.left_panel_x
self.stats_y = self.skills_y + 170 # Уменьшен отступ для статистики из-за меньшего
шрифта

self.next_x = self.right_panel_x
self.next_y = self.grid_y + 20

self.grid = [[0 for _ in range(self.grid_width)] for _ in range(self.grid_height)]
self.current_piece = None
self.next_pieces = []
self.hold_piece = None
self.can_hold = True
self.score = 0
self.level = 1
self.lines_cleared = 0
self.fall_speed = 0.5
self.fall_time = 0
self.soft_drop_speed = 0.05

self.mana = 0
self.max_mana = 100
self.mana_per_line = 10
self.mana_per_landing = 3
self.slow_time_remaining = 0
self.slow_time_duration = 4.0
self.original_fall_speed = self.fall_speed

self.skills = {
    "replace": {"name": "Заменить", "cost": 55, "key": pygame.K_1},
    "slow_time": {"name": "Замедление", "cost": 70, "key": pygame.K_2},
    "skip": {"name": "Пропуск", "cost": 40, "key": pygame.K_3}
}

```

```
self.move_delay = 0.15
self.move_timer = 0
self.last_move_direction = None
```

```
self.combo_count = 0
self.last_clear_time = 0
self.combo_texts = []
self.score_popups = []
```

```
self.shape_history = []
```

```
self.music_playing = False
self.music_loaded = False
self.game_over_music_loaded = False
self.remove_sound_loaded = False
self.game_over_music_playing = False
```

```
self.block_textures = self.load_block_textures()
```

```
self.shapes = [
    [[1, 1, 1, 1]],
    [[1, 1], [1, 1]],
    [[1, 1, 1], [0, 1, 0]],
    [[1, 1, 1], [1, 0, 0]],
    [[1, 1, 1], [0, 0, 1]],
    [[0, 1, 1], [1, 1, 0]],
    [[1, 1, 0], [0, 1, 1]]
]
```

```
self.shape_textures = [0, 1, 2, 3, 4, 5, 6]
```

```
self.init_game()
self.load_music()
```

```
def load_music(self):
    try:
        current_dir = os.path.dirname(os.path.abspath(__file__))
```

```
possible_paths = [
    os.path.join(current_dir, "static", "music.mp3"),
    os.path.join(current_dir, "music.mp3"),
    os.path.join(current_dir, "..", "static", "music.mp3"),
    os.path.join(current_dir, "..", "music.mp3"),
    "static/music.mp3",
    "../static/music.mp3",
    "../static/music.mp3",
    "music.mp3",
    "../music.mp3"
]
```

```
music_path = None
for path in possible_paths:
    if os.path.exists(path):
        music_path = path
        break
```

```
if music_path:
    pygame.mixer.music.load(music_path)
    pygame.mixer.music.set_volume(0.7)
    pygame.mixer.music.play(-1)
    self.music_playing = True
    self.music_loaded = True
else:
    self.music_loaded = False
```

```
except:
    self.music_loaded = False
```

```
def load_sound_effects(self):
    try:
        current_dir = os.path.dirname(os.path.abspath(__file__))

        remove_paths = [
            os.path.join(current_dir, "static", "remove.mp3"),
            os.path.join(current_dir, "remove.mp3"),
            os.path.join(current_dir, "..", "static", "remove.mp3"),
```

```
    "static/remove.mp3",  
    "./static/remove.mp3",  
    "../static/remove.mp3"  
]
```

```
remove_path = None  
for path in remove_paths:  
    if os.path.exists(path):  
        remove_path = path  
        break
```

```
if remove_path:  
    self.remove_sound = pygame.mixer.Sound(remove_path)  
    self.remove_sound.set_volume(0.5)  
    self.remove_sound_loaded = True  
else:  
    self.remove_sound_loaded = False
```

```
gameover_paths = [  
    os.path.join(current_dir, "static", "gameover.mp3"),  
    os.path.join(current_dir, "gameover.mp3"),  
    os.path.join(current_dir, "..", "static", "gameover.mp3"),  
    "static/gameover.mp3",  
    "./static/gameover.mp3",  
    "../static/gameover.mp3"  
]
```

```
gameover_path = None  
for path in gameover_paths:  
    if os.path.exists(path):  
        gameover_path = path  
        break
```

```
if gameover_path:  
    self.game_over_music_loaded = True  
    self.game_over_music_path = gameover_path  
else:  
    self.game_over_music_loaded = False
```

```

except:
    self.remove_sound_loaded = False
    self.game_over_music_loaded = False

def play_remove_sound(self):
    if self.remove_sound_loaded:
        self.remove_sound.play()

def play_game_over_music(self):
    if self.game_over_music_playing:
        return

    if self.game_over_music_loaded:
        try:
            pygame.mixer.music.stop()
            pygame.mixer.music.load(self.game_over_music_path)
            pygame.mixer.music.set_volume(0.7)
            pygame.mixer.music.play(-1)
            self.game_over_music_playing = True
            self.music_playing = True
        except:
            pass

def stop_music(self):
    if self.music_loaded or self.game_over_music_loaded:
        pygame.mixer.music.stop()
        self.music_playing = False
        self.game_over_music_playing = False

def restart_music(self):
    if self.music_loaded:
        try:
            pygame.mixer.music.stop()
            current_dir = os.path.dirname(os.path.abspath(__file__))
            music_path = os.path.join(current_dir, "static", "music.mp3")
            if os.path.exists(music_path):
                pygame.mixer.music.load(music_path)

```

```
        pygame.mixer.music.set_volume(0.7)
        pygame.mixer.music.play(-1)
        self.music_playing = True
        self.game_over_music_playing = False
    except:
        pass
```

```
def toggle_music(self):
    if not self.music_loaded and not self.game_over_music_loaded:
        return

    if self.music_playing:
        pygame.mixer.music.pause()
        self.music_playing = False
    else:
        pygame.mixer.music.unpause()
        self.music_playing = True
```

```
def init_game(self):
    self.grid = [[0 for _ in range(self.grid_width)] for _ in range(self.grid_height)]
    self.current_piece = None
    self.next_pieces = []
    self.hold_piece = None
    self.can_hold = True
    self.score = 0
    self.level = 1
    self.lines_cleared = 0
    self.fall_speed = 0.5
    self.fall_time = 0
    self.game_over = False
    self.combo_count = 0
    self.combo_texts = []
    self.score_popups = []
    self.shape_history = []
    self.game_over_music_playing = False

    self.mana = 0
    self.max_mana = 100
```

```

self.slow_time_remaining = 0
self.original_fall_speed = self.fall_speed

self.load_sound_effects()
self.restart_music()

for _ in range(4):
    self.next_pieces.append(self.create_random_piece())

self.spawn_piece()

def load_block_textures(self):
    textures = {}

    current_dir = os.path.dirname(os.path.abspath(__file__))

    empty_texture = pygame.Surface((self.block_size, self.block_size))
    empty_texture.fill((0, 0, 0))
    textures[0] = empty_texture

    texture_files = [
        "ice.png",
        "sand.png",
        "crystal.png",
        "fire.png",
        "bolt.png",
        "plant.png",
        "rock.png"
    ]

    for i, filename in enumerate(texture_files, start=1):
        texture_paths = [
            os.path.join(current_dir, "static", filename),
            os.path.join(current_dir, filename),
            os.path.join(current_dir, "..", "static", filename),
            f"static/{filename}",
            f"./static/{filename}",
            f"..static/{filename}"

```

```
]
```

```
texture_loaded = False
for path in texture_paths:
    if os.path.exists(path):
        try:
            image = pygame.image.load(path).convert_alpha()
            texture = pygame.transform.scale(image, (self.block_size, self.block_size))
            textures[i] = texture
            texture_loaded = True
            break
        except:
            continue
```

```
if not texture_loaded:
    color = self.get_color_for_texture(i)
    texture = pygame.Surface((self.block_size, self.block_size))
    texture.fill(color)
    pygame.draw.rect(texture, (50, 50, 50), texture.get_rect(), 1)
    textures[i] = texture
```

```
return textures
```

```
def get_color_for_texture(self, texture_index):
    colors = {
        1: (0, 255, 255),
        2: (255, 255, 0),
        3: (128, 0, 128),
        4: (255, 165, 0),
        5: (0, 0, 255),
        6: (0, 255, 0),
        7: (255, 0, 0)
    }
    return colors.get(texture_index, (255, 255, 255))
```

```
def create_random_piece(self):
    available_shapes = list(range(len(self.shapes)))
```

```

if len(self.shape_history) >= 2:
    last_two = self.shape_history[-2:]
    if last_two[0] == last_two[1]:
        if last_two[0] in available_shapes:
            available_shapes.remove(last_two[0])

if not available_shapes:
    available_shapes = list(range(len(self.shapes)))

shape_idx = random.choice(available_shapes)
shape = self.shapes[shape_idx]

self.shape_history.append(shape_idx)
if len(self.shape_history) > 4:
    self.shape_history.pop(0)

x = self.grid_width // 2 - len(shape[0]) // 2
return {
    'shape': shape,
    'texture': self.shape_textures[shape_idx] + 1,
    'x': x,
    'y': 0
}

def spawn_piece(self):
    if not self.next_pieces:
        for _ in range(3):
            self.next_pieces.append(self.create_random_piece())

    if self.next_pieces:
        self.current_piece = self.next_pieces.pop(0)
        self.next_pieces.append(self.create_random_piece())
        self.can_hold = True

    if self.current_piece['y'] == 0 and len(self.shape_history) > 1:
        self.add_mana(self.mana_per_landing)

```

```

        if self.check_collision(self.current_piece['x'], self.current_piece['y'],
self.current_piece['shape']):
            self.game_over = True
            if not self.game_over_music_playing:
                self.play_game_over_music()

def rotate_piece(self, piece):
    shape = piece['shape']
    rotated = [[shape[y][x] for y in range(len(shape)-1, -1, -1)] for x in range(len(shape[0]))]
    return rotated

def check_collision(self, x, y, shape):
    if not shape:
        return True

    for row_idx, row in enumerate(shape):
        for col_idx, cell in enumerate(row):
            if cell:
                if (x + col_idx < 0 or x + col_idx >= self.grid_width or
                    y + row_idx >= self.grid_height):
                    return True
                if y + row_idx >= 0 and self.grid[y + row_idx][x + col_idx]:
                    return True
    return False

def adjust_position_for_rotation(self, x, y, original_shape, rotated_shape):
    original_width = len(original_shape[0])
    original_height = len(original_shape)
    rotated_width = len(rotated_shape[0])
    rotated_height = len(rotated_shape)

    if x + rotated_width > self.grid_width:
        x = self.grid_width - rotated_width

    if x < 0:
        x = 0

    if y + rotated_height > self.grid_height:

```

```
y = self.grid_height - rotated_height
```

```
return x, y
```

```
def merge_piece(self):
```

```
    if not self.current_piece:
```

```
        return
```

```
    shape = self.current_piece['shape']
```

```
    texture = self.current_piece['texture']
```

```
    for row_idx, row in enumerate(shape):
```

```
        for col_idx, cell in enumerate(row):
```

```
            if cell and 0 <= self.current_piece['y'] + row_idx < self.grid_height:
```

```
                if 0 <= self.current_piece['x'] + col_idx < self.grid_width:
```

```
                    self.grid[self.current_piece['y'] + row_idx][self.current_piece['x'] + col_idx] =
```

```
texture
```

```
self.clear_lines()
```

```
self.spawn_piece()
```

```
def clear_lines(self):
```

```
    lines_to_clear = []
```

```
    for row_idx in range(self.grid_height):
```

```
        if all(self.grid[row_idx]):
```

```
            lines_to_clear.append(row_idx)
```

```
    if lines_to_clear:
```

```
        self.play_remove_sound()
```

```
    lines_to_clear.sort(reverse=True)
```

```
    for row_idx in lines_to_clear:
```

```
        del self.grid[row_idx]
```

```
    for _ in range(len(lines_to_clear)):
```

```
        self.grid.insert(0, [0 for _ in range(self.grid_width)])
```

```

        cleared = len(lines_to_clear)
        self.update_score(cleared)

        self.add_mana(cleared * self.mana_per_line)

        self.create_clear_effects(cleared)

def update_score(self, lines_cleared):
    base_scores = {1: 100, 2: 300, 3: 500, 4: 800}
    base_score = base_scores.get(lines_cleared, 1000)

    combo_multiplier = 1 + (self.combo_count * 0.5)
    score_earned = int(base_score * self.level * combo_multiplier)

    current_time = pygame.time.get_ticks()
    if current_time - self.last_clear_time < 2000:
        self.combo_count += 1
    else:
        self.combo_count = 1

    self.last_clear_time = current_time

    self.score += score_earned
    self.lines_cleared += lines_cleared
    self.level = self.lines_cleared // 10 + 1
    self.fall_speed = max(0.05, 0.5 - (self.level - 1) * 0.05)
    self.original_fall_speed = self.fall_speed

    self.score_popups.append({
        'text': f"+{score_earned}",
        'x': self.grid_x + self.grid_width * self.block_size // 2,
        'y': self.grid_y + self.grid_height * self.block_size // 2,
        'timer': 2.0,
        'color': self.get_score_color(lines_cleared)
    })

def create_clear_effects(self, lines_cleared):
    combo_texts = {

```

```

1: ["Good!"],
2: ["Great!", "Double!"],
3: ["Awesome!", "Triple!"],
4: ["EXCELLENT!", "QUAD!", "AMAZING!"]
}

```

```

texts = combo_texts.get(lines_cleared, ["PERFECT!"])
if self.combo_count > 1:
    texts.append(f"COMBO x{self.combo_count}!")

```

```

for i, text in enumerate(texts):
    y_offset = 100 + (i * 40)
    if "COMBO" in text:
        y_offset += 60

```

```

self.combo_texts.append({
    'text': text,
    'x': self.grid_x + self.grid_width * self.block_size // 2,
    'y': self.grid_y + y_offset,
    'timer': 3.0,
    'color': self.get_combo_color(lines_cleared)
})

```

```

def get_score_color(self, lines):
    colors = {
        1: (100, 100, 255),
        2: (100, 255, 100),
        3: (255, 255, 100),
        4: (255, 100, 255)
    }
    return colors.get(lines, (255, 255, 255))

```

```

def get_combo_color(self, lines):
    colors = {
        1: (100, 200, 255),
        2: (100, 255, 200),
        3: (255, 200, 100),
        4: (255, 100, 255)
    }

```

```

    }

    return colors.get(lines, (255, 255, 100))

def hard_drop(self):
    if not self.current_piece:
        return

    while not self.check_collision(self.current_piece['x'], self.current_piece['y'] + 1,
self.current_piece['shape']):
        self.current_piece['y'] += 1
    self.merge_piece()

def hold_current_piece(self):
    if not self.current_piece or not self.can_hold:
        return

    if self.hold_piece:
        current_shape = self.current_piece['shape']
        current_texture = self.current_piece['texture']

        self.current_piece['shape'] = self.hold_piece['shape']
        self.current_piece['texture'] = self.hold_piece['texture']
        self.current_piece['x'] = self.grid_width // 2 - len(self.current_piece['shape'][0]) // 2
        self.current_piece['y'] = 0

        self.hold_piece['shape'] = current_shape
        self.hold_piece['texture'] = current_texture
    else:
        self.hold_piece = {
            'shape': self.current_piece['shape'],
            'texture': self.current_piece['texture']
        }
        self.spawn_piece()

    self.can_hold = False

def add_manana(self, amount):
    self.manana = min(self.max_manana, self.manana + amount)

```

```

def use_skill(self, skill_key):
    if self.paused or self.game_over:
        return

    skill_name = None
    for name, skill_data in self.skills.items():
        if skill_data["key"] == skill_key:
            skill_name = name
            break

    if not skill_name:
        return

    skill_cost = self.skills[skill_name]["cost"]

    if self.mana < skill_cost:
        return

    self.mana -= skill_cost

    if skill_name == "replace":
        self.use_replace_skill()
    elif skill_name == "slow_time":
        self.use_slow_time_skill()
    elif skill_name == "skip":
        self.use_skip_skill()

def use_replace_skill(self):
    if not self.hold_piece or not self.current_piece:
        return

    new_piece = {
        'shape': self.hold_piece['shape'][:,],
        'texture': self.hold_piece['texture'],
        'x': self.grid_width // 2 - len(self.hold_piece['shape'][0]) // 2,
        'y': 0
    }

```

```

    if not self.check_collision(new_piece['x'], new_piece['y'], new_piece['shape']):
        self.current_piece = new_piece

def use_slow_time_skill(self):
    self.slow_time_remaining = self.slow_time_duration
    self.fall_speed = self.original_fall_speed * 3.0

def use_skip_skill(self):
    self.spawn_piece()

def handle_continuous_movement(self, dt):
    if self.paused or self.game_over or not self.current_piece:
        return

    keys = pygame.key.get_pressed()
    current_direction = None

    if keys[pygame.K_LEFT]:
        current_direction = 'left'
    elif keys[pygame.K_RIGHT]:
        current_direction = 'right'

    if current_direction:
        if current_direction != self.last_move_direction:
            self.move_timer = 0
            self.last_move_direction = current_direction
            self.move_piece(current_direction)
        else:
            self.move_timer += dt
            if self.move_timer >= self.move_delay:
                self.move_piece(current_direction)
                self.move_timer = 0.1
    else:
        self.last_move_direction = None
        self.move_timer = 0

def move_piece(self, direction):

```

```

        if direction == 'left':
            if not self.check_collision(self.current_piece['x'] - 1, self.current_piece['y'],
self.current_piece['shape']):
                self.current_piece['x'] -= 1
        elif direction == 'right':
            if not self.check_collision(self.current_piece['x'] + 1, self.current_piece['y'],
self.current_piece['shape']):
                self.current_piece['x'] += 1

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                self.running = False
                return "quit"
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    self.stop_music()
                    return "menu"
                elif event.key == pygame.K_p:
                    self.paused = not self.paused
                    if self.paused and self.music_playing and (self.music_loaded or
self.game_over_music_loaded):
                        pygame.mixer.music.pause()
                    elif not self.paused and self.music_playing and (self.music_loaded or
self.game_over_music_loaded):
                        pygame.mixer.music.unpause()
                elif event.key == pygame.K_m:
                    self.toggle_music()
                elif event.key == pygame.K_r and self.game_over:
                    self.init_game()
                elif event.key in [pygame.K_1, pygame.K_2, pygame.K_3]:
                    self.use_skill(event.key)
                elif not self.paused and not self.game_over and self.current_piece:
                    if event.key == pygame.K_UP:
                        rotated = self.rotate_piece(self.current_piece)

                        if not self.check_collision(self.current_piece['x'], self.current_piece['y'], rotated):
                            self.current_piece['shape'] = rotated

```

```

        else:
            new_x, new_y = self.adjust_position_for_rotation(
                self.current_piece['x'],
                self.current_piece['y'],
                self.current_piece['shape'],
                rotated
            )

            if not self.check_collision(new_x, new_y, rotated):
                self.current_piece['x'] = new_x
                self.current_piece['y'] = new_y
                self.current_piece['shape'] = rotated

        elif event.key == pygame.K_SPACE:
            self.hard_drop()
        elif event.key == pygame.K_c:
            self.hold_current_piece()

    return "tetris"

def update(self, dt):
    if self.paused or not self.running or self.game_over or not self.current_piece:
        return

    if self.slow_time_remaining > 0:
        self.slow_time_remaining -= dt
        if self.slow_time_remaining <= 0:
            self.slow_time_remaining = 0
            self.fall_speed = self.original_fall_speed

    self.handle_continuous_movement(dt)

    self.fall_time += dt
    current_speed = self.soft_drop_speed if pygame.key.get_pressed()[pygame.K_DOWN]
    else self.fall_speed

    if self.fall_time >= current_speed:
        self.fall_time = 0

```

```

        if not self.check_collision(self.current_piece['x'], self.current_piece['y'] + 1,
self.current_piece['shape']):
            self.current_piece['y'] += 1
        else:
            self.merge_piece()

self.update_effects(dt)

def update_effects(self, dt):
    for effect in self.combo_texts[:]:
        effect['timer'] -= dt
        if effect['timer'] <= 0:
            self.combo_texts.remove(effect)
        else:
            effect['y'] -= dt * 30

    for popup in self.score_popups[:]:
        popup['timer'] -= dt
        if popup['timer'] <= 0:
            self.score_popups.remove(popup)
        else:
            popup['y'] -= dt * 50

def draw_grid(self):
    grid_rect = pygame.Rect(
        self.grid_x - 2,
        self.grid_y - 2,
        self.grid_width * self.block_size + 4,
        self.grid_height * self.block_size + 4
    )
    pygame.draw.rect(self.screen, self.grid_color, grid_rect)

    for y in range(self.grid_height):
        for x in range(self.grid_width):
            rect = pygame.Rect(
                self.grid_x + x * self.block_size,
                self.grid_y + y * self.block_size,
                self.block_size, self.block_size

```

)

```
if self.grid[y][x]:
    self.screen.blit(self.block_textures[self.grid[y][x]], rect)
else:
    pygame.draw.rect(self.screen, self.background_color, rect)
    pygame.draw.rect(self.screen, self.grid_color, rect, 1)
```

```
def draw_piece(self, piece, x, y, alpha=255):
```

```
    if not piece:
```

```
        return
```

```
    shape = piece['shape']
```

```
    texture_idx = piece['texture']
```

```
    for row_idx, row in enumerate(shape):
```

```
        for col_idx, cell in enumerate(row):
```

```
            if cell:
```

```
                rect = pygame.Rect(
                    x + col_idx * self.block_size,
                    y + row_idx * self.block_size,
                    self.block_size, self.block_size
```

```
                )
```

```
                texture = self.block_textures[texture_idx].copy()
```

```
                if alpha < 255:
```

```
                    texture.set_alpha(alpha)
```

```
                self.screen.blit(texture, rect)
```

```
def draw_ghost_piece(self):
```

```
    if not self.current_piece:
```

```
        return
```

```
    ghost_y = self.current_piece['y']
```

```
    while not self.check_collision(self.current_piece['x'], ghost_y + 1,
self.current_piece['shape']):
```

```
        ghost_y += 1
```

```
    if ghost_y != self.current_piece['y']:
```

```

ghost_piece = {
    'shape': self.current_piece['shape'],
    'texture': self.current_piece['texture']
}
self.draw_piece(ghost_piece,
                self.grid_x + self.current_piece['x'] * self.block_size,
                self.grid_y + ghost_y * self.block_size,
                alpha=100)

def draw_mana_bar(self):
    bar_width = 200
    bar_height = 25
    bar_rect = pygame.Rect(self.mana_x, self.mana_y, bar_width, bar_height)

    pygame.draw.rect(self.screen, self.grid_color, bar_rect)

    fill_width = int((self.mana / self.max_mana) * bar_width)
    fill_rect = pygame.Rect(self.mana_x, self.mana_y, fill_width, bar_height)
    pygame.draw.rect(self.screen, self.mana_color, fill_rect)

    pygame.draw.rect(self.screen, self.text_color, bar_rect, 2)

    mana_text = self.score_font.render(f"Ма́на: {self.mana}/{self.max_mana}", True,
self.text_color)
    self.screen.blit(mana_text, (self.mana_x, self.mana_y - 25))

def draw_skills(self):
    skill_title = self.score_font.render("НАВЫ́КИ", True, self.text_color)
    self.screen.blit(skill_title, (self.skills_x, self.skills_y - 25))

    skill_bg = pygame.Rect(self.skills_x - 10, self.skills_y - 10, 210, 150)
    pygame.draw.rect(self.screen, self.grid_color, skill_bg)

    y_offset = 0
    for skill_name, skill_data in self.skills.items():
        skill_y = self.skills_y + y_offset

        skill_available = self.mana >= skill_data["cost"]

```

```

text_color = self.skill_available_color if skill_available else self.skill_unavailable_color

skill_text = self.skill_font.render(f'{{skill_data['name']}} ({{skill_data['cost']}})', True,
text_color)
self.screen.blit(skill_text, (self.skills_x, skill_y))

key_number = str(list(self.skills.keys()).index(skill_name) + 1)
key_text = self.score_font.render(f'[{key_number}]', True, text_color)
self.screen.blit(key_text, (self.skills_x + 160, skill_y))

if skill_name == "slow_time" and self.slow_time_remaining > 0:
    time_left = max(0, self.slow_time_remaining)
    time_text = self.score_font.render(f'{{time_left:.1f}}s', True, (255, 255, 100))
    self.screen.blit(time_text, (self.skills_x + 130, skill_y))

y_offset += 40

def draw_stats(self):
    """Отрисовка панели статистики с уменьшенным шрифтом"""
    stats_bg = pygame.Rect(self.stats_x - 10, self.stats_y - 10, 210, 190) # Уменьшил высоту
    pygame.draw.rect(self.screen, self.grid_color, stats_bg)

    stats_title = self.stats_font.render("СТАТИСТИКА", True, self.text_color) # Используем
stats_font
    self.screen.blit(stats_title, (self.stats_x, self.stats_y - 20))

    y_offset = 0
    spacing = 22 # Уменьшенный интервал между строками

    # Отображаем всю статистику уменьшенным шрифтом
    score_text = self.stats_font.render(f'Очки: {{self.score}}', True, self.text_color)
    self.screen.blit(score_text, (self.stats_x, self.stats_y + y_offset))
    y_offset += spacing

    level_text = self.stats_font.render(f'Уровень: {{self.level}}', True, self.text_color)
    self.screen.blit(level_text, (self.stats_x, self.stats_y + y_offset))
    y_offset += spacing

```

```

lines_text = self.stats_font.render(f"Линии: {self.lines_cleared}", True, self.text_color)
self.screen.blit(lines_text, (self.stats_x, self.stats_y + y_offset))
y_offset += spacing

combo_text = self.stats_font.render(f"Комбо: x{self.combo_count}", True,
self.text_color)
self.screen.blit(combo_text, (self.stats_x, self.stats_y + y_offset))
y_offset += spacing

slow_text = self.stats_font.render(f"Замедление:", True, self.text_color)
self.screen.blit(slow_text, (self.stats_x, self.stats_y + y_offset))
slow_status = self.stats_font.render(f"'Активно' if self.slow_time_remaining > 0 else
'Неактивно'",
True, self.accent_color if self.slow_time_remaining > 0 else
self.text_color)
self.screen.blit(slow_status, (self.stats_x + 95, self.stats_y + y_offset))
y_offset += spacing

music_text = self.stats_font.render(f"Музыка:", True, self.text_color)
self.screen.blit(music_text, (self.stats_x, self.stats_y + y_offset))
music_status = "Вкл" if self.music_playing else "Выкл"
if not self.music_loaded and not self.game_over_music_loaded:
    music_status = "Нет"
music_status_text = self.stats_font.render(f"{music_status} [M]", True, self.text_color)
self.screen.blit(music_status_text, (self.stats_x + 65, self.stats_y + y_offset))
y_offset += spacing

def draw_sidebar(self):
    # HOLD панель
    hold_text = self.score_font.render("HOLD", True, self.text_color)
    self.screen.blit(hold_text, (self.hold_x, self.hold_y - 40))
    hold_bg = pygame.Rect(self.hold_x - 10, self.hold_y - 20, 200, 180)
    pygame.draw.rect(self.screen, self.grid_color, hold_bg)

    if self.hold_piece:
        shape_width = len(self.hold_piece['shape'][0]) * self.block_size
        shape_height = len(self.hold_piece['shape']) * self.block_size
        hold_x_centered = self.hold_x + (200 - shape_width) // 2

```

```

        hold_y_centered = self.hold_y + (180 - shape_height) // 2
        self.draw_piece(self.hold_piece, hold_x_centered, hold_y_centered)

# NEXT панель
next_text = self.score_font.render("NEXT", True, self.text_color)
self.screen.blit(next_text, (self.next_x, self.next_y - 40))
next_bg = pygame.Rect(self.next_x - 10, self.next_y - 20, 200, 260)
pygame.draw.rect(self.screen, self.grid_color, next_bg)

for i, piece in enumerate(self.next_pieces[:3]):
    shape_width = len(piece['shape'][0]) * self.block_size
    shape_height = len(piece['shape']) * self.block_size
    next_x_centered = self.next_x + (200 - shape_width) // 2
    next_y_centered = self.next_y + i * 80 + (80 - shape_height) // 2
    self.draw_piece(piece, next_x_centered, next_y_centered)

# Рисуем все панели
self.draw_mana_bar()
self.draw_skills()
self.draw_stats()

def draw_effects(self):
    for effect in self.combo_texts:
        alpha = min(255, int(effect['timer'] * 255))
        text_surface = self.combo_font.render(effect['text'], True, effect['color'])
        text_surface.set_alpha(alpha)
        text_rect = text_surface.get_rect(center=(effect['x'], effect['y']))
        self.screen.blit(text_surface, text_rect)

    for popup in self.score_popups:
        alpha = min(255, int(popup['timer'] * 255))
        text_surface = self.info_font.render(popup['text'], True, popup['color'])
        text_surface.set_alpha(alpha)
        text_rect = text_surface.get_rect(center=(popup['x'], popup['y']))
        self.screen.blit(text_surface, text_rect)

def draw_pause_screen(self):
    overlay = pygame.Surface((self.screen_width, self.screen_height), pygame.SRCALPHA)

```

```

overlay.fill((0, 0, 0, 128))
self.screen.blit(overlay, (0, 0))

pause_text = self.title_font.render("PAUSED", True, self.accent_color)
pause_rect = pause_text.get_rect(center=(self.screen_width//2, self.screen_height//2 - 50))
self.screen.blit(pause_text, pause_rect)

instr_text = self.info_font.render("Press P to continue", True, self.text_color)
instr_rect = instr_text.get_rect(center=(self.screen_width//2, self.screen_height//2 + 50))
self.screen.blit(instr_text, instr_rect)

def draw_game_over_screen(self):
    overlay = pygame.Surface((self.screen_width, self.screen_height), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 128))
    self.screen.blit(overlay, (0, 0))

    game_over_text = self.title_font.render("GAME OVER", True, (255, 50, 50))
    game_over_rect = game_over_text.get_rect(center=(self.screen_width//2,
self.screen_height//2 - 50))
    self.screen.blit(game_over_text, game_over_rect)

    score_text = self.info_font.render(f"Final Score: {self.score}", True, self.text_color)
    score_rect = score_text.get_rect(center=(self.screen_width//2, self.screen_height//2 + 20))
    self.screen.blit(score_text, score_rect)

    mana_text = self.info_font.render(f"Final Mana: {self.mana}", True, self.text_color)
    mana_rect = mana_text.get_rect(center=(self.screen_width//2, self.screen_height//2 + 60))
    self.screen.blit(mana_text, mana_rect)

    restart_text = self.info_font.render("Press R to restart", True, self.text_color)
    restart_rect = restart_text.get_rect(center=(self.screen_width//2, self.screen_height//2 +
110))
    self.screen.blit(restart_text, restart_rect)

    menu_text = self.info_font.render("Press ESC for menu", True, self.text_color)
    menu_rect = menu_text.get_rect(center=(self.screen_width//2, self.screen_height//2 +
160))
    self.screen.blit(menu_text, menu_rect)

```

```

def draw(self):
    self.screen.fill(self.background_color)

    self.draw_grid()
    if self.current_piece:
        self.draw_ghost_piece()
        self.draw_piece(self.current_piece,
                        self.grid_x + self.current_piece['x'] * self.block_size,
                        self.grid_y + self.current_piece['y'] * self.block_size)

    self.draw_sidebar()
    self.draw_effects()

    if self.paused:
        self.draw_pause_screen()
    elif self.game_over:
        self.draw_game_over_screen()

    pygame.display.flip()

def run(self):
    last_time = pygame.time.get_ticks()

    while self.running:
        current_time = pygame.time.get_ticks()
        dt = (current_time - last_time) / 1000.0
        last_time = current_time

        result = self.handle_events()
        if result != "tetris":
            return result

        self.update(dt)
        self.draw()
        self.clock.tick(self.fps)

    return "menu"

```

```
def main():
    pygame.init()
    # Увеличиваем размеры окна для отдельного запуска
    game = TetrisGame(1000, 800)
    result = game.run()

    if result == "menu":
        print("Возврат в меню")

    pygame.quit()
    sys.exit()

if __name__ == "__main__":
    main()
```